

BSc (Hons) Computer Science
BSc (Hons) Cyber Security
Module Code: QH0305 Module Title: Problem Solving
Assessment Sheet 7

Instructions:

This is one of the eight assessment tasks which will contribute to the overall mark. You will need to complete the tasks as outlined below and then document them in a Word document file. As a minimum, you should provide screenshots of the following:

- Your code
- The output that your code generates

In instances where your code could generate different outputs depending on what values are given, you should provide multiple screenshots of the console screen, showing the different outputs in order to demonstrate that the code works correctly.

Your Word document should have appropriate headings to ensure that each task can easily be identified alongside the rest of your work.

This assessment will focus on the use of functions and file handling.

You must attempt all tasks on this sheet to achieve a higher grade. For example, if you want to gain marks between 70 - 100, you must complete all other grades first and add them to your portfolio with screenshots.

A zip folder with all Grade codes (in .c or .txt format) must be submitted alongside the portfolio (MS Word file). This is in addition to the code presented as screenshots.

Assessment Task 7: Media Lab Equipment Loan Manager

Scenario:

Build a small system to manage **equipment loans** in a university Media Lab. The system should let staff **add**, **view**, **search**, **update**, and **delete** equipment records, with **file persistence** so data is retained between runs. The solution must be organised using **functions** and **basic file I/O**, with functionality and complexity increasing across the grade bounds.

To achieve Grade D (Between 40-49):

1. Define a struct named Item to store:
 - **id** (integer)
 - **assetTag** (string, e.g., camera tag "CAM-104")
 - **status** (string or single char, e.g., A=Available, L=Loaned)
2. Create the data file (plain text, e.g., loans.txt) if it doesn't exist.
3. Implement no-parameter, no-return functions (handle input/output inside):

addItem() → get **id**, **assetTag**, **status** from the user and append to the file.
displayItems() → read all records from the file and print them in a tidy format.

4. Demonstrate adding a few (3 or more) items and then displaying the list.

To achieve Grade C (Between 50-59):

1. Refactor functions to use parameters where appropriate:
 - *addItem*(int id, char assetTag[], char status[])
 - keep a *displayItems*() with no parameters
2. Add a function *searchItemById*(int id) that reads the file and displays the matching record (or a message if not found).
3. Duplicate-ID check: modify *addItem*(...) to refuse adding if an item with the same id already exists. If so, then print a friendly message and don't append the file.

To achieve Grade B (Between 60-69):

1. Refactor your code to now introduce return values, making functions informative
 - int *addItem*(...) → return 1 if added, 0 if duplicate.
 - int *searchItemById*(int id) → return the record index/position if found, -1 otherwise.
2. Implement int *updateItem*(int id, const char newAssetTag[], const char newStatus[]):
 - Read all records into memory (array of Item type), apply the update by id (modifying assetTag and status), then rewrite the file.
 - Return 1 if successful, 0 if record with that id is not found.
3. Ensure the file content reflects updates (old data replaced by new).

To achieve Grade A (Between 70-100):

1. Implement deletion with file rewrite:
 - int *deleteItem*(int id) → load all records into suitable array, remove the target record, then overwrite the file with the remaining items; return 1 if deleted, 0 if not found.
2. File persistence in program flow:
 - On start, attempt to load data from file into an in-memory array.
 - Provide a simple menu (add / view / search / update / delete / save / exit).
 - On save or exit, write current in-memory data to the file.
3. Variable scope: keep a small set of global constants/variables (e.g., the filename, array capacity), and use local variables for function internals. In your portfolio, add a concise note explaining local vs global scope in your implementation.

Assignment Preparation Guidelines

- All components of the assignment report must be Word-processed (**handwritten text or hand drawn diagrams are not acceptable**), font size must be within the range of 11 point to 14 point including the headings, body text and any texts within diagrams.
- Standard and commonly used fonts such as Times New Roman, Arial or Calibri should be used.
- All figures, graphs and tables must be numbered and labelled with short explanations.
- Material from external sources must be properly acknowledged and cited within the text using the Harvard referencing system.
- All components of the assignment (text, diagrams, code etc.) and all assessment sheets must be submitted in one Word file.
- The report should be logically structured, the core of the report may start by defining the problem / requirements, followed by the proposed solution including a detailed discussion, analysis and evaluation, leading to the implementation and testing stage, finally a conclusion and personal reflection on learning.
- Screenshots without description / discussion are not suitable as they do not express your understanding or support your work adequately.

Submission instructions

- This is a portfolio assignment with eight tasks in total. Each task will be completed and saved in the portfolio. Once the portfolio is completed, it should be submitted on Turnitin. The submission link to Turnitin can be found under the “Assessment Tab” in your module section in the SOL VLE.
- Please note file size limitation might apply. Your report must be under 250MB.
- The source code (in .c or .txt format) for each task should be **zipped** and complete single ZIP file **submitted** to the VLE alongside the report.
- The Assignment Brief can be found under the “Assessment Tab” in your module section in the SOL VLE.
- **Refer to the Assignment Brief** to find the links to Late Submissions, Extenuating Circumstances, Academic Misconduct, Ethics Policy, Grade marking and Guidance for online submission through Solent Online Learning (SOL).